A Hopf Algebra on Proof Trees for a Nonmonotonic Sequent Calculus

Abstract

We define algebraic structure on proof trees arising from a nonmonotonic multi-succedent sequent calculus. The primitive combinatorial operation is a matching-leaf grafting operation, which composes derivations by replacing a leaf of one proof tree with another proof tree having the same concluding sequent. This yields a natural candidate pre-Lie product on proof trees. A second, cut-based operation yields a coproduct encoding decomposition into subderivations. Since raw address-level witness counting contains bureaucratic artefacts, we pass to a quotient identifying proof presentations that differ only by inessential scheduling data. At the quotient level we obtain the pre-Lie identity and hence, via a standard enveloping construction, a Hopf algebra of proof trees for a nonmonotonic sequent calculus.

1 Introduction

The present paper develops an algebraic treatment of inference in a nonmonotonic setting. Our guiding idea is that proof trees should carry both a *compositional* and a *decompositional* structure. Composition corresponds to grafting one derivation into another, while decomposition corresponds to extracting subderivations by admissible cuts. The resulting interaction is naturally Hopf-algebraic.

The proof-theoretic background is a nonmonotonic sequent calculus. The failure of weakening is essential: if inference is not stable under arbitrary enlargement of the premise set, then proof composition cannot be treated as a free combinatorial operation. Instead, composition must respect logical interfaces, here formalized by matching conclusion sequents.

This gives a proof-theoretically constrained analogue of the familiar combinatorial Hopf algebras on rooted trees. The trees considered here are not abstract rooted trees, but proof trees generated from a sequent calculus with

a primitive base consequence relation. Their logical decoration is therefore constitutive of the algebraic structure rather than external to it.

Our eventual aim is to show that, after quotienting away proof-theoretically inessential bureaucracy, the grafting operation satisfies the pre-Lie identity. This yields a Hopf algebra of derivations in the sense relevant to nonmonotonic inference. In this way we treat logical constants and derivation patterns not merely as syntactic devices but as algebraically organised features of inference itself.

2 The Nonmonotonic Sequent Calculus

2.1 Formulas and sequents

We begin with a simple propositional language.

Definition 1. *Let* At *be a set of atomic symbols. The set* $\mathcal{L}$ *of formulas is generated by the grammar*

$$A ::= p \mid \bot \mid (A \Rightarrow A) \mid (A \wedge A) \mid (A \vee A) \mid \neg A,$$

where $p \in \mathsf{At}$.

The constant $\bot$ plays a special role as an incompatibility marker. At this stage we do not impose any global principle identifying derivability of $\bot$ with inconsistency via a deduction theorem. Rather, incompatibility is tracked proof-theoretically through the primitive consequence relation.

Definition 2. *A* multi-succedent sequent *is an expression*

$$\Gamma \mid\!\sim \Delta,$$

where Γ *and* Δ *are finite multisets of formulas.*

The use of multisets reflects that structural rules are not built in by default. In particular, we do not assume weakening.

2.2 Primitive consequence relation

Definition 3. *A* base consequence relation *is a relation*

$$\mathsf{base} \subseteq \mathsf{MSet}(\mathcal{L}) \times \mathsf{MSet}(\mathcal{L}).$$

Equivalently, for multisets Γ, Θ *of formulas, one may write* $\mathsf{base}(\Gamma, \Theta)$.

The base relation supplies the leaf-level proof-theoretic data. It should be thought of as the primitive inferential background over which the sequent rules generate derivations.

2.3 Derivations

Fix a base consequence relation base. We define a derivability relation inductively.

Definition 4. *The derivations of the nonmonotonic multi-succedent sequent calculus* $\mathsf{NMMS}(\mathsf{base})$ *are generated by the following clauses.*

Base axiom. For any multisets Γ, Θ,

$$\frac{\mathsf{base}(\Gamma, \Theta)}{\Gamma \mid\!\sim \Theta} \quad \text{(Base)}$$

Implication rules.

$$\frac{\Gamma \mid\!\sim A, \Delta \qquad B, \Gamma \mid\!\sim \Delta}{(A \Rightarrow B), \Gamma \mid\!\sim \Delta} \quad (\Rightarrow L)$$

$$\frac{A, \Gamma \mid\!\sim B, \Delta}{\Gamma \mid\!\sim (A \Rightarrow B), \Delta} \quad (\Rightarrow R)$$

Conjunction rules.

$$\frac{A, B, \Gamma \mid\!\sim \Delta}{(A \wedge B), \Gamma \mid\!\sim \Delta} \quad (\wedge L)$$

$$\frac{\Gamma \mid\!\sim A, \Delta \qquad \Gamma \mid\!\sim B, \Delta}{\Gamma \mid\!\sim (A \wedge B), \Delta} \quad (\wedge R)$$

Disjunction rules.

$$\frac{A, \Gamma \mid\!\sim \Delta \qquad B, \Gamma \mid\!\sim \Delta}{(A \vee B), \Gamma \mid\!\sim \Delta} \quad (\vee L)$$

$$\frac{\Gamma \mid\!\sim A, B, \Delta}{\Gamma \mid\!\sim (A \vee B), \Delta} \quad (\vee R)$$

Negation rules.

$$\frac{\Gamma \mid\!\sim A, \Delta}{\neg A, \Gamma \mid\!\sim \Delta} \quad (\neg L) \qquad \frac{A, \Gamma \mid\!\sim \Delta}{\Gamma \mid\!\sim \neg A, \Delta} \quad (\neg R)$$

Remark 5. *No weakening rule is assumed. In particular, the rules*

$$\frac{\Gamma \mid\!\sim \Delta}{\Gamma, A \mid\!\sim \Delta} \qquad \frac{\Gamma \mid\!\sim \Delta}{\Gamma \mid\!\sim \Delta, A}$$

are not taken to be admissible in general. This is the formal expression of the nonmonotonic character of the system.

Remark 6. *Likewise, we do not assume unrestricted cut-elimination as part of the background. The algebraic structures developed below are intended precisely for a setting in which composition and decomposition of derivations must be handled in a controlled, proof-theoretically sensitive way.*

3 Proof Trees

3.1 Plain proof trees

Definition 7. *A* plain proof tree *is a rooted tree whose nodes are labelled by sequents and whose internal nodes are further tagged by one of the rule symbols*

$$\mathsf{Base},\ \Rightarrow L,\ \Rightarrow R,\ \wedge L,\ \wedge R,\ \vee L,\ \vee R,\ \neg L,\ \neg R.$$

Leaves carry only a concluding sequent.

Thus a proof tree forgets the specific derivation object while retaining the inferential shape of the derivation, including the sequent labelling at each node and the rule applied at each interior node.

Definition 8. *If t is a proof tree, we write:*

- $\mathrm{conc}(t)$ *for its concluding sequent,*
- $\mathrm{size}(t)$ *for its number of nodes,*
- $\mathrm{ht}(t)$ *for its height,*
- $\mathrm{children}(t)$ *for its immediate subtrees.*

Definition 9. *An* address *in a proof tree is a finite list of natural numbers, indicating a path from the root to a subtree.*

Addresses allow us to refer to a specific subtree or leaf of a proof tree. They play a technical role in defining local grafting operations, though the final algebraic constructions will quotient away inessential address-level bureaucracy.

3.2 From derivations to proof trees

Every derivation in $\mathsf{NMMS}(\mathsf{base})$ determines a proof tree by forgetting the derivation witness and retaining only the rule-labelled tree of sequents.

Definition 10. *Let d be a derivation of a sequent s. The associated proof tree*

$$\mathrm{toTree}(d)$$

is defined recursively by replacing each inference step by the corresponding rule-labelled node and each base derivation by a leaf.

Concretely, if d ends with an implication-left step, then $\mathrm{toTree}(d)$ has root label $(A \Rightarrow B), \Gamma \mathrel{|\!\sim} \Theta$ and two children given by the proof trees of the premiss derivations; similarly for the other rules.

Lemma 11. *For every derivation d of a sequent s, the conclusion of its associated proof tree is s:*

$$\mathrm{conc}(\mathrm{toTree}(d)) = s.$$

4 Matching-Leaf Grafting

4.1 Local grafting

The basic compositional operation in our framework is the replacement of a leaf of one proof tree by another proof tree with the same concluding sequent.

Definition 12. *Let u and t be proof trees, and let a be an address in t. We say that a is a* graftable leaf address for u in t *if the subtree of t at address a is a leaf labelled by the sequent* $\mathrm{conc}(u)$.

Definition 13. *Suppose a is a graftable leaf address for u in t. The* matching-leaf grafting of u into t at a, *denoted*

$$\mathrm{graft}_a(u, t),$$

is the proof tree obtained from t by replacing the leaf at address a by the tree u.

Equivalently, grafting is defined only when the conclusion of the inserted tree matches the sequent labelling the target leaf.

Definition 14. *The set of all matching-leaf graftings of u into t is*

$$\mathrm{Graft}(u, t) = \{\, \mathrm{graft}_a(u, t) \mid a \text{ is a graftable leaf address for } u \text{ in } t \,\}.$$

4.2 Closure on derivation trees

A crucial proof-theoretic fact is that grafting derivation trees does not leave the world of derivation trees.

Theorem 15 (Closure of matching-leaf grafting under derivability)**.** *Let* d_{out} *and* d_{in} *be derivations in* $\mathsf{NMMS}(\mathsf{base})$*, with proof trees* $\text{toTree}(d_{\text{out}})$ *and* $\text{toTree}(d_{\text{in}})$*. If* a *is a graftable leaf address for* $\text{toTree}(d_{\text{in}})$ *in* $\text{toTree}(d_{\text{out}})$*, then there exists a derivation* d' *such that*

$$\text{graft}_a\big(\text{toTree}(d_{\text{in}}), \text{toTree}(d_{\text{out}})\big) = \text{toTree}(d').$$

Remark 16. *This theorem justifies reading matching-leaf grafting as genuine proof composition rather than a merely combinatorial tree operation. The operation respects the inductive generation of derivations by the sequent rules.*

5 Two-Step Grafting and the Geometry of the Associator

We now analyse iterated matching-leaf grafting. This is the structural core of the pre-Lie analysis, since the pre-Lie identity is controlled by the difference between the two iterated products

$$x \triangleright (y \triangleright z) \qquad \text{and} \qquad (x \triangleright y) \triangleright z.$$

At the raw tree level, the relevant combinatorics is governed by the interaction of two graft addresses inside a common ambient tree.

5.1 Dependent and independent two-step graftings

Suppose first that y is grafted into z at an address a, producing a tree z', and then x is grafted into z' at an address b, producing a tree w. There are only two possibilities: either the second graft lands inside the subtree previously inserted at a, or it takes place at an address independent of a.

Theorem 17 (Full two-step graft decomposition)**.** *Let* x, y, z *be proof trees. Suppose that grafting* y *into* z *at an address* a *yields a tree* z'*, and grafting* x *into* z' *at an address* b *yields a tree* w*. Then one of the following two geometric situations occurs.*

(i) ***Nested case.*** *There exist an address* c *and a proof tree* y' *such that*

$$b = a \frown c,$$

and such that grafting x into y at c yields y', while grafting y' into z at a yields w.

(ii) ***Independent case.*** *There exists a proof tree z_3 such that the addresses a and b are incomparable, grafting x into z at b yields z_3, and grafting y into z_3 at a yields w.*

Remark 18. *The theorem says that every successful two-step grafting factors either through a genuine insertion of x into the already-inserted tree y, or through two independent insertions into disjoint regions of z. This dichotomy is the proof-tree analogue of the familiar decomposition of rooted-tree insertions into nested and disjoint contributions.*

5.2 Commutation of independent graftings

The independent case admits a stronger formulation: incomparable successful grafts commute.

Theorem 19 (Commutation of incomparable graftings)**.** *Let x, y, z be proof trees, and let a, b be incomparable addresses in z. Suppose that grafting y into z at a produces z_1, and grafting x into z_1 at b produces z_2. Suppose also that grafting x into z at b produces z_3. Then grafting y into z_3 at a is defined and yields the same final tree:*

$$\mathrm{graft}_a(y, z_3) = z_2.$$

Remark 20. *This is the precise sense in which independent proof insertions commute. The result does not say that grafting is globally commutative; rather, it says that noninteracting local insertions can be permuted without changing the resulting proof tree.*

6 The Free Abelian Group on Proof Trees and the Raw Grafting Product

We now pass from individual proof trees to formal linear combinations of proof trees.

Definition 21. *Let $\mathbb{Z}\langle \mathsf{PTree} \rangle$ denote the free abelian group on the set of proof trees. For a proof tree t, write $[t]$ for the corresponding basis element.*

Definition 22 (Tree-level grafting product)**.** *For proof trees u and t, define*

$$u \triangleright t := \sum_{v \in \mathrm{Graft}(u,t)} [v] \ \in \ \mathbb{Z}\langle \mathsf{PTree} \rangle.$$

Equivalently, $u \triangleright t$ is the formal sum of all proof trees obtained by grafting u into t at a matching leaf.

This is the primitive product from which the pre-Lie structure will be obtained after quotienting the appropriate witness data.

Proposition 23 (Compatibility with generators)**.** *The bilinear grafting product is determined on basis elements by the tree-level grafting operation. In particular, for proof trees u and t,*

$$[u] \triangleright [t] = u \triangleright t.$$

Remark 24. *This is formally straightforward, but mathematically important: the product on the free abelian group is not an additional structure but the linear extension of matching-leaf grafting on individual proof trees.*

6.1 Iterated grafting and coefficient formulas

The next step is to express iterated grafting in terms of counts of two-step graftings.

Proposition 25 (Expansion of left action over a graft sum)**.** *For proof trees u, t, w,*

$$u \triangleright (t \triangleright w) = \sum_{z' \in \mathrm{Graft}(t,w)} u \triangleright z'.$$

Proposition 26 (Coefficient formula for $x \triangleright (y \triangleright z)$)**.** *Let x, y, z, w be proof trees. Then the coefficient of $[w]$ in $x \triangleright (y \triangleright z)$ is the number of two-step graftings of the following form: first graft y into z, obtaining some intermediate tree z', and then graft x into z', obtaining w. Explicitly,*

$$[w]\big(x \triangleright (y \triangleright z)\big) = \#\Big\{ z' \in \mathrm{Graft}(y, z) \ \Big| \ w \in \mathrm{Graft}(x, z') \Big\}.$$

Proposition 27 (Coefficient formula for $(x \triangleright y) \triangleright z$)**.** *Let x, y, z, w be proof trees. Then the coefficient of $[w]$ in $(x \triangleright y) \triangleright z$ is the number of two-step graftings of the following form: first graft x into y, obtaining some intermediate tree y', and then graft y' into z, obtaining w. Explicitly,*

$$[w]\big((x \triangleright y) \triangleright z\big) = \#\Big\{ y' \in \mathrm{Graft}(x, y) \ \Big| \ w \in \mathrm{Graft}(y', z) \Big\}.$$

Remark 28. *These coefficient formulas show that the pre-Lie identity is fundamentally a comparison between two families of two-step grafting witnesses. The remainder of the argument therefore focuses on organising, comparing, and quotienting those witnesses.*

7 The Coproduct Side

Alongside the grafting product, the theory also carries a decomposition operation coming from admissible cuts of proof trees.

Definition 29. *For each proof tree t, let $\Delta(t)$ denote the collection of all decompositions of t arising from admissible cuts. Each such cut yields a pair consisting of a forest of extracted subtrees and a remainder tree.*

The precise admissibility conditions on cuts were introduced elsewhere in the formal development; for the present discussion it is enough to note that the empty cut is always admissible.

Proposition 30 (Nonemptiness of the coproduct data)**.** *For every proof tree t, the coproduct data associated with t is nonempty. Equivalently, $\Delta(t)$ always contains at least one term.*

Idea of proof. The empty cut is admissible. Its corresponding decomposition contributes the trivial term consisting of the empty forest together with the original tree t. □

Remark 31. *This is a minimal but conceptually useful fact: the decomposition structure is never vacuous. In particular, the cut-based coalgebra has a canonical trivial term, just as in the usual rooted-tree coproducts.*

8 Two-Step Codes and Bureaucratic Equivalence

The coefficient formulas above reduce the pre-Lie problem to a comparison of two-step grafting data. At raw address level, however, this data contains bureaucratic artefacts: the same underlying proof-theoretic interaction may be presented by different address choices or by different orders of independent insertions.

To remove this noise we introduce an equivalence relation on two-step witness codes.

8.1 Two-step codes

Definition 32. *Fix proof trees x, y, z, w. A* two-step code *for the production of w from x, y, z records one of the possible geometric shapes of a successful two-step grafting. In particular, one distinguishes:*

- *left-outer codes,*
- *right-outer codes,*
- *left-inner codes,*
- *right-inner codes.*

These encode whether one starts from the left- or right-associated iterated product, and whether the second insertion is independent of or nested inside the first.

Remark 33. *The exact coding carries address-level witness data and intermediate trees. That detail is important for the formal proof, but for the mathematical presentation the essential point is that two-step codes classify successful two-step graftings by their geometric type.*

8.2 Equivalence generated by commuting outer contributions

Definition 34 (Two-step equivalence)**.** *Fix proof trees x, y, z, w. The relation*

$$\sim_{x,y,z,w}$$

on two-step codes is the least equivalence relation generated by the following identifications.

(1) *A left-outer code is identified with any right-outer code representing the same proof-theoretic contribution obtained by commuting independent outer insertions.*

(2) *A left-outer code is identified with any right-inner code representing the same final contribution after transport across the corresponding two-step geometry.*

(3) *Symmetrically, a right-outer code is identified with any left-outer code obtained by commuting independent outer insertions.*

(4) *Symmetrically, a right-outer code is identified with any left-inner code representing the same contribution after the corresponding transport.*

Remark 35. *This relation is designed to identify exactly those two-step presentations that differ only by inessential bureaucratic data. The generators correspond to the basic proof-theoretic symmetries discovered in the analysis of two-step grafting.*

Definition 36. *The* two-step quotient *for x, y, z, w is the set of equivalence classes of two-step codes under $\sim_{x,y,z,w}$.*

8.3 Existence of opposite-side representatives

One of the first structural consequences of the quotient construction is that an outer witness on one side always determines some representative on the opposite side.

Lemma 37 (Left-outer classes admit right-side representatives)**.** *Let x, y, z, w be proof trees, and let a successful left-outer two-step grafting from (x, y, z) to w be given. Then there exists a two-step code on the right side such that the original left-outer code is equivalent to it in the two-step quotient.*

Remark 38. *This lemma is the beginning of the comparison theorem between the two parenthesisations. It does not yet assert uniqueness or purity, but it shows that left-associated outer contributions are never isolated: each is linked by the quotient to some right-associated contribution.*

9 Towards the Class-Level Pre-Lie Identity

The quotient construction is introduced because raw witness-counting does not yet isolate the genuinely proof-theoretic content of the associator. Independent insertions may be represented by several bureaucratically distinct address patterns, and these must be identified before a clean algebraic identity emerges.

At the quotient level, the intended picture is that every contribution class in the associator decomposes into three sectors:

(1) quotient-trivial noise,

(2) pure outer residual classes,

(3) pure inner residual classes.

The decisive further step, to be developed in the next stage of the paper, is to prove that mixed residual classes disappear, so that the remaining quotient classes are pure and typed. This allows one to compare the two parenthesisations class by class and derive the quotient-level pre-Lie identity.

10 Hopf-Algebraic Outlook

Once the quotient-level pre-Lie identity has been established, one may pass from the primitive pre-Lie structure to an enveloping Hopf algebra by a standard construction of Oudom–Guin type. In parallel, the admissible-cut decomposition of proof trees supplies a coproduct reflecting the internal subderivation structure of proofs.

The resulting object should therefore be understood as a Hopf algebra of nonmonotonic proof trees: the algebra structure models composition of proofs, while the coalgebra structure models decomposition into subproofs. This is the sense in which the present framework seeks to capture both the constructive and the analytic aspects of inference within a single algebraic object.

11 Conclusion

We have assembled the proof-theoretic and combinatorial foundations for a Hopf algebraic treatment of derivations in a nonmonotonic sequent calculus. The basic objects are proof trees generated from a primitive base consequence relation. Their local composition is given by matching-leaf grafting, and the analysis of iterated grafting leads naturally to a quotient on two-step witness data identifying bureaucratically equivalent proof presentations.

What remains is to complete the structural analysis of quotient classes, prove the class-level pre-Lie identity, and integrate this with the cut-based coproduct in a full Hopf algebra construction. Even at the present stage, however, the framework already displays the central idea: nonmonotonic inference admits a rich algebraic organisation in which proof composition and proof decomposition are treated on an equal footing.